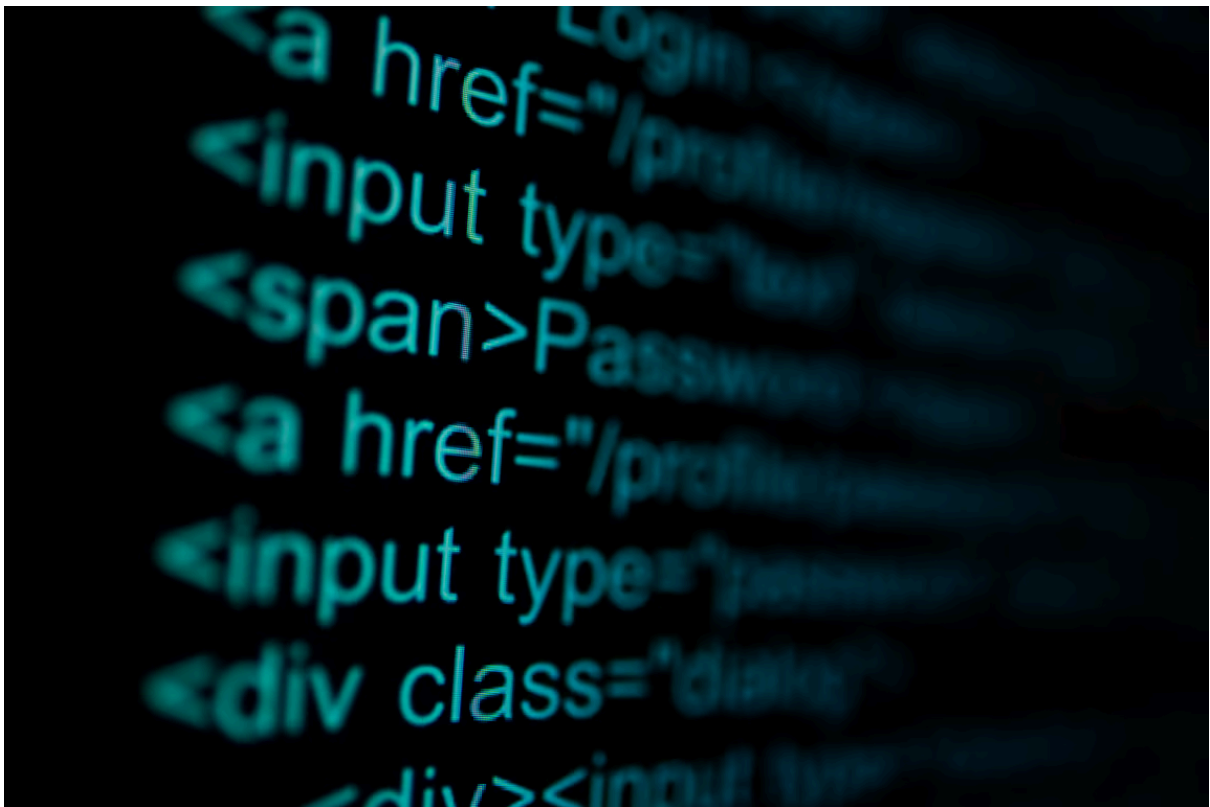


Unidad 1: Introducción a la Programación con Python

Introducción a la Programación

Breve Historia de la Programación

La historia de la programación está fuertemente vinculada a la aparición y crecimiento exponencial de los computadores. En sus inicios, programar implicaba el uso de código binario, con cadenas de 0s y 1s, el lenguaje que entiende directamente la computadora, lo cual era sumamente complejo. Más tarde, se creó el lenguaje ensamblador, que en vez de números utilizaba letras, algo más fácil de recordar.



Finalmente, surgieron los lenguajes de alto nivel, que suelen utilizar términos en inglés para dictar las órdenes a seguir, actuando como un intermediario entre el lenguaje de la máquina (binario) y el “código fuente” (procesado por un compilador o intérprete).

Lenguajes de programación más conocidos

Lenguajes de Frontend

Los lenguajes de frontend son aquellos que interactúan directamente con el usuario en la interfaz del cliente. Los principales lenguajes de frontend incluyen:

- JavaScript
- HTML y CSS (aunque HTML y CSS no son lenguajes de programación, son esenciales para la estructura y el estilo de los sitios web)

Lenguajes de Backend

Los lenguajes de backend son aquellos que gestionan la lógica del servidor, bases de datos y las interacciones del servidor. Los principales lenguajes de backend incluyen:

- Python
 - Java
 - Javascript con Node Js
 - Ruby
 - PHP
-

Diferencia entre lenguajes de Frontend y Backend

Frontend

El frontend se refiere a la parte de un sitio web que interactúa directamente con los usuarios, en los dispositivos que utilicen para consumir un servicio web (normalmente un navegador o app móvil), también conocido como el lado del cliente. Involucra la maquetación lógica visible de un sitio web y es lo que los usuarios pueden ver e interactuar.

Backend

El backend se refiere a la parte del sitio web que conecta con la base de datos y el servidor, también conocido como el lado del servidor. Involucra el desarrollo de APIs del lado del servidor y bases de datos, y se encarga de conectar la base de datos con la interfaz de la web o aplicación.

Tareas involucradas en el Backend

- Creación, integración y mantenimiento de bases de datos
- Comunicación del servidor, aplicación y base de datos

- Creación, mantenimiento, testeo y debugging del backend
 - Creación de backups de bases de datos
 - Conversión y devolución de datos
 - Mantenimiento del sistema y deploy
 - Integración con APIs
 - Seguridad
-

Introducción a Python

Python como Lenguaje de Backend

Python es un lenguaje de programación de código abierto, orientado a objetos, simple y fácil de entender. Tiene una sintaxis sencilla y cuenta con una vasta biblioteca de herramientas, lo que lo hace un lenguaje único. Python es un lenguaje interpretado, con tipado dinámico, multiparadigma y multiplataforma. En este curso, nos enfocaremos en Python como un lenguaje de backend.

Ventajas y Desventajas de Python

Ventajas:

- Fácil de aprender
- Fácil de usar
- Orientado a objetos
- Software libre
- Portable
- Multiplataforma
- Poderoso

Desventajas:

- Lentitud en multithread
- Lentitud de ejecución
- Tipado poco estricto

Python es ampliamente utilizado en diversas áreas como inteligencia artificial, big data, data science, frameworks de pruebas y desarrollo web. En este curso, nos centraremos en la parte web del desarrollo.

Conclusión

La programación es un proceso fundamental para crear aplicaciones y software, utilizando diversos lenguajes que permiten materializar conceptos y proyectos. Python, como uno de los lenguajes más versátiles y accesibles, se convierte en una herramienta esencial para el desarrollo backend, ofreciendo numerosas ventajas para programadores de todos los niveles.

Primeros pasos con Python y variables

Variables en Python

Las variables en Python son etiquetas que referencian datos almacenados en la memoria del programa. A diferencia de otros lenguajes de programación donde las variables son cajas que contienen datos, en Python, las variables son simplemente nombres asignados a objetos que contienen los datos.

Definición y uso de variables en Python:

1. **Definir una variable:** Para crear una variable en Python, simplemente asigna un valor a un nombre utilizando el operador de asignación (=).

```
```python
nombre = "Juan"
edad = 25

```
```

En este ejemplo, `nombre` es una variable que contiene la cadena de texto "Juan", y `edad` es una variable que contiene el número entero 25.

2. Reglas para nombrar variables:

- Los nombres de las variables deben comenzar con una letra (a-z, A-Z) o un guion bajo (_).
- El resto del nombre puede incluir letras, números (0-9) y guiones bajos.
- Python distingue entre mayúsculas y minúsculas, por lo que `edad` y `Edad` son variables diferentes.
- Es una buena práctica usar nombres de variables descriptivos y utilizar la convención `snake\_case`, donde las palabras están separadas por guiones bajos.

```
```python
fecha_de_nacimiento = "27 de octubre de 1997"

```
```

```
'''
```

3. **Asignación múltiple:** Python permite asignar valores a múltiples variables en una sola línea.

```
'''python
a, b, c = 1, 2, 3

'''
```

4. **Intercambiar valores:** Puedes intercambiar los valores de dos variables sin usar una variable temporal.

```
'''python
a, b = b, a

'''
```

5. **Entrada del usuario:** Utiliza la función `input()` para obtener datos del usuario y almacenarlos en una variable.

```
'''python
nombre = input("Ingrese su nombre: ")

'''
```

6. **Buenas prácticas para nombrar variables:**

- Usa nombres claros y significativos que describan el contenido o el propósito de la variable.
- Evita usar palabras reservadas de Python como nombres de variables (por ejemplo, `print`, `for`, `while`).

```
'''python
contador_de_usuarios = 0

'''
```

Ejemplos prácticos:

1. **Definir y usar una variable:**

```
'''python
mensaje = "Hola, mundo!"
print(mensaje)

'''
```

2. Actualizar el valor de una variable:

```
```python
x = 5
x = x + 2
print(x) # Imprime 7
```
```

3. Operaciones con variables:

```
```python
a = 10
b = 20
suma = a + b
print(suma) # Imprime 30
```
```

4. Uso de variables con diferentes tipos de datos:

```
```python
nombre = "Ana"
edad = 30
altura = 1.75
es_estudiante = True
```
```

Resumen de buenas prácticas:

- Elige nombres de variables que sean descriptivos y fáciles de entender.
- Usa la convención `snake\_case` para separar palabras en los nombres de variables.
- Evita el uso de palabras reservadas como nombres de variables.
- Asigna valores claros y específicos a las variables para mejorar la legibilidad del código.

Siguiendo estas recomendaciones, podrás definir y utilizar variables en Python de manera efectiva y clara, facilitando la escritura y el mantenimiento de tu código.

Operaciones básicas y cadenas de texto en Python

En Python, las operaciones aritméticas básicas son fundamentales para realizar cálculos y manipular datos numéricos. Las principales operaciones aritméticas son suma, resta, multiplicación y división. A continuación, se describen estas operaciones con ejemplos.

Suma (+)

La suma en Python se realiza utilizando el operador `+`.

```
```python
a = 5
b = 3
resultado = a + b # resultado es 8
```
```

Resta (-)

La resta se lleva a cabo con el operador `-`.

```
```python
a = 5
b = 3
resultado = a - b # resultado es 2
```
```

Multiplicación (*)

La multiplicación se realiza con el operador `*`.

```
```python
a = 5
b = 3
resultado = a * b # resultado es 15
```
```

División (/)

La división se efectúa usando el operador `/`. Este operador siempre devuelve un número de punto flotante.

```
```python
a = 5
b = 3
resultado = a / b # resultado es aproximadamente 1.6667
...

```

## División entera (`//`)

Si se desea obtener el cociente entero de una división, se utiliza el operador `//`.

```
```python
a = 5
b = 3
resultado = a // b # resultado es 1
...

```

Módulo (`%`)

El operador de módulo (`%`) devuelve el residuo o resto de la división de dos números.

```
```python
a = 5
b = 3
resultado = a % b # resultado es 2
...

```

## Potencia (`\*\*`)

Para realizar potencias, se utiliza el operador `\*\*`.

```
```python
a = 5
b = 3
resultado = a ** b # resultado es 125
...

```

Precedencia de Operadores

La precedencia de operadores en Python sigue reglas específicas, similar a las matemáticas. Estas reglas determinan el orden en que se evalúan las expresiones cuando hay más de un operador presente.

1. **Paréntesis** `()` : Tienen la más alta precedencia y se utilizan para forzar el orden de evaluación.
 2. **Exponentes** `\*\*` : Tienen la segunda mayor precedencia.
 3. **Multiplicación** `\*`, **División** `/`, **División entera** `//`, **Módulo** `%` : Estos operadores tienen la misma precedencia y se evalúan de izquierda a derecha.
 4. **Suma** `+` y **Resta** `-` : Tienen la menor precedencia y se evalúan de izquierda a derecha.
-

Ejemplo de precedencia de operadores:

```
```python
resultado = (4 + 8) / 2 * 5 ** 2 - (9 + 3) / 2

Paso 1: (4 + 8) = 12
Paso 2: 5 ** 2 = 25
Paso 3: 12 / 2 = 6
Paso 4: 6 * 25 = 150
Paso 5: (9 + 3) = 12
Paso 6: 12 / 2 = 6
Paso 7: 150 - 6 = 144
resultado es 144
```
```

Ejercicios Prácticos

1. Suma de dos números:

```
```python
a = int(input("Ingrese el primer número: "))
b = int(input("Ingrese el segundo número: "))
print("La suma es:", a + b)

```
```

2. Calcular el área de un rectángulo:

```
```python
base = float(input("Ingrese la base del rectángulo: "))
altura = float(input("Ingrese la altura del rectángulo: "))
area = base * altura
print("El área del rectángulo es:", area)

```
```

Con esta introducción, se puede empezar a realizar cálculos básicos en Python, entendiendo cómo funcionan las operaciones aritméticas y la precedencia de los operadores.

Cadenas de Texto en Python

Creación de Cadenas de Texto:

En Python, las cadenas de texto (strings) se crean utilizando comillas simples (' ') o dobles (" "). Ambas formas son equivalentes y se pueden usar indistintamente. Ejemplos:

```
```python
cadena1 = 'Hola, mundo'
cadena2 = "Python es genial"

```
```

Concatenación de Cadenas:

La concatenación de cadenas se refiere a unir dos o más cadenas en una sola. En Python, esto se hace utilizando el operador `+`. Ejemplo:

```
```python
cadena1 = 'Hola'
cadena2 = 'Mundo'
cadena_completa = cadena1 + ' ' + cadena2
print(cadena_completa) # Salida: Hola Mundo

```
```

Manipulación de Cadenas:

Python ofrece varias funciones y métodos para manipular cadenas de texto. Algunas de las operaciones más comunes incluyen:

1. Longitud de una Cadena:

Utiliza la función `len()` para obtener la longitud de una cadena.

```
```python
```

```
cadena = 'Python'
print(len(cadena)) # Salida: 6

'''
```

## 2. Indexación:

Puedes acceder a caracteres individuales de una cadena utilizando índices. Los índices comienzan desde 0.

```
'''python
cadena = 'Python'
print(cadena[0]) # Salida: P
print(cadena[-1]) # Salida: n (índice negativo cuenta desde el final)

'''
```

## 3. Rebanado (Slicing):

Puedes obtener una subcadena utilizando la notación `[inicio:fin:paso]`. El carácter en la posición `fin` no está incluido.

```
'''python
cadena = 'Python'
print(cadena[1:6:2]) # Salida: yhn (empieza en la posición 1 y salta cada 2 caracteres)
Si se omite el paso, toma el valor 1 por defecto
print(cadena[0:2]) # Salida: Py
Si se omite el final, se toma hasta el último
print(cadena[2:]) # Salida: thon
Si se omite el inicio, se toma desde el inicio
print(cadena[:4]) # Salida: Pyth

'''
```

## 4. Métodos de Cadenas:

Python proporciona varios métodos incorporados para manipular cadenas, como `upper()`, `lower()`, `replace()`, `find()`, entre otros.

```
'''python
cadena = 'Hola, Mundo'
print(cadena.upper()) # Salida: HOLA, MUNDO
print(cadena.lower()) # Salida: hola, mundo
print(cadena.replace('Hola', 'Adiós')) # Salida: Adiós, Mundo
print(cadena.find('Mundo')) # Salida: 6 (posición inicial de la subcadena)

'''
```

---

# Ejercicios Prácticos

## 1. Concatenación:

Crea dos variables de cadena con tu nombre y tu apellido. Concátelas y muestra el resultado.

```
```python
nombre = 'Juan'
apellido = 'Pérez'
nombre_completo = nombre + ' ' + apellido
print(nombre_completo) # Salida: Juan Pérez

```
```

## 2. Rebanado:

Dada la cadena `cadena = "Programación con Python"`, utiliza el rebanado para extraer la palabra "Python".

```
```python
cadena = "Programación con Python"
subcadena = cadena[18:]
print(subcadena) # Salida: Python

```
```

## 3. Manipulación:

Dada la cadena `cadena = "Python es divertido"`, convierte toda la cadena a mayúsculas, reemplaza la palabra "divertido" por "fácil" y encuentra la posición de la palabra "es".

```
```python
cadena = "Python es divertido"
print(cadena.upper()) # Salida: PYTHON ES DIVERTIDO
nueva_cadena = cadena.replace('divertido', 'fácil')
print(nueva_cadena) # Salida: Python es fácil
posicion = cadena.find('es')
print(posicion) # Salida: 7

```
```

Estos ejemplos y ejercicios te ayudarán a familiarizarte con el trabajo con cadenas de texto en Python. Practica estos conceptos para mejorar tus habilidades en programación con Python.

---

# Introducción a números y cadenas de caracteres

## Números

Los números en Python se dividen en varios tipos:

- Enteros (int): Son números sin parte decimal, tanto positivos como negativos.
- Reales (float): Son números con decimales.
- Complejos (complex): Son números que incluyen una parte real y una imaginaria.

Las operaciones básicas que se pueden realizar con números incluyen:

- Suma (+): Adición de dos números.
- Resta (-): Sustracción de un número de otro.
- Multiplicación (\*): Producto de dos números.
- División (/): Cociente de dos números.
- División entera (//): Cociente entero de la división de dos números.
- Módulo (%): Resto de la división de dos números.
- Potenciación (): Un número elevado a la potencia de otro.

También se explorará la **precedencia de operadores**, que es el orden en que se evalúan las operaciones en una expresión matemática.

## Cadenas de Caracteres

Las cadenas de caracteres son secuencias de caracteres que se utilizan para representar texto. En Python, se pueden definir utilizando comillas simples (' ') o dobles (" ").

Algunas operaciones básicas con cadenas incluyen:

- Concatenación (+): Unión de dos o más cadenas.
- Repetición (\*): Repetición de una cadena un número determinado de veces.
- Indexación: Acceso a caracteres específicos dentro de una cadena utilizando índices.
- Slicing: Extracción de subcadenas utilizando notaciones de inicio y fin de índices.
- Longitud (len): Determinación del número de caracteres en una cadena.

Se cubrirán también funciones integradas de Python para manipular cadenas, como:

- `upper()`: Convierte todos los caracteres de una cadena a mayúsculas.
  - `lower()`: Convierte todos los caracteres de una cadena a minúsculas.
  - `replace()`: Reemplaza una subcadena por otra dentro de una cadena.
  - `split()`: Divide una cadena en una lista de subcadenas basadas en un delimitador.
- 

## Tipos de número y operaciones numéricas en Python

En Python, existen tres tipos principales de números que podemos utilizar en nuestras operaciones y cálculos: enteros, números reales (de punto flotante) y números complejos. A continuación, describimos cada uno de ellos con ejemplos:

### Enteros

Los números enteros son aquellos que no tienen decimales. Pueden ser positivos, negativos o cero. En Python, los enteros se representan con el tipo ``int``.

Ejemplos:

```
```python
1
-10
0
523
-817
```
```

Python 3 permite que los números enteros sean tan grandes como la memoria del ordenador lo permita, sin un límite máximo.

---

### Números Reales (Punto Flotante)

Los números reales, también conocidos como números de punto flotante, son aquellos que tienen una parte decimal. En Python, se representan con el tipo ``float``. Además, Python ofrece el tipo ``Decimal`` para representar fracciones de forma más precisa.

Ejemplos:

```
```python
0.5
-3.14
2.71828
-0.0001
```
```

---

## Números Complejos

Los números complejos son una extensión del sistema numérico que permite la existencia de raíces cuadradas de números negativos. Un número complejo tiene una parte real y una parte imaginaria. En Python, se representan con el tipo `complex`, utilizando la letra `j` para denotar la parte imaginaria.

Ejemplos:

```
```python
1+2j
-3+4j
0-1j
5j
```
```

Los números complejos son útiles en diversas ramas de la matemática y la física, especialmente en la teoría de circuitos eléctricos y la mecánica cuántica.

---

## Ejemplos de uso

A continuación, se presentan algunos ejemplos prácticos de cómo se pueden utilizar estos tipos de números en Python:

```
```python
# Enteros
a = 5
b = -3
print(a + b) # Resultado: 2

# Números de punto flotante
x = 3.14
y = 2.71
print(x * y) # Resultado: 8.5094
```
```

```
Números complejos
z1 = 1 + 2j
z2 = 3 - 4j
print(z1 + z2) # Resultado: (4-2j)
```

...

Cada tipo de número tiene su propio conjunto de operaciones y propiedades, lo que los hace adecuados para diferentes tipos de problemas y cálculos. Con esta variedad de tipos numéricos, Python proporciona una gran flexibilidad para trabajar con números en diversas aplicaciones.

---

## Operaciones numéricas en Python

Python proporciona una variedad de operadores aritméticos que se utilizan para realizar cálculos matemáticos. A continuación, se detallan las operaciones aritméticas básicas y avanzadas, con ejemplos para cada una.

### Suma

La suma se realiza utilizando el operador `+`. Este operador se utiliza para sumar dos números.

Ejemplo:

```
```python
a = 10
b = 5
resultado = a + b
print(resultado) # Resultado: 15
```

...

Resta

La resta se realiza utilizando el operador `-`. Este operador se utiliza para restar un número de otro.

Ejemplo:

```
```python
a = 10
b = 5
resultado = a - b
```



```
print(resultado) # Resultado: 5
```

```
'''
```

## Multiplicación

La multiplicación se realiza utilizando el operador `*`. Este operador se utiliza para multiplicar dos números.

Ejemplo:

```
'''python
a = 10
b = 5
resultado = a * b
print(resultado) # Resultado: 50
```

```
'''
```

## División

La división se realiza utilizando el operador `/`. Este operador se utiliza para dividir un número por otro.

Ejemplo:

```
'''python
a = 10
b = 5
resultado = a / b
print(resultado) # Resultado: 2.0
```

```
'''
```

## División Entera

La división entera se realiza utilizando el operador `//`. Este operador divide un número por otro y devuelve la parte entera del resultado.

Ejemplo:

```
'''python
a = 10
b = 3
resultado = a // b
print(resultado) # Resultado: 3
```

```
'''
```

## Módulo

El operador de módulo `%` devuelve el resto de la división de un número por otro.

Ejemplo:

```
```python
a = 10
b = 3
resultado = a % b
print(resultado) # Resultado: 1
```
```

## Potencia

La potencia se realiza utilizando el operador `\*\*`. Este operador se utiliza para elevar un número a la potencia de otro.

Ejemplo:

```
```python
a = 2
b = 3
resultado = a ** b
print(resultado) # Resultado: 8
```
```

---

## Precedencia de los Operadores

La precedencia de los operadores determina el orden en que se evalúan las operaciones en una expresión. La siguiente es la jerarquía de precedencia de los operadores aritméticos en Python, de mayor a menor:

1. `()`: Paréntesis
2. `\*\*`: Potencia
3. `\*`, `/`, `//`, `%`: Multiplicación, División, División Entera, Módulo
4. `+`, `-`: Suma, Resta

Ejemplo:

```
```python
resultado = (2 + 3) * 4 ** 2 / (5 - 1) + 6
print(resultado) # Resultado: 26.0
```
```

...

En este ejemplo, la expresión dentro de los paréntesis se evalúa primero, seguida de la potencia, la multiplicación/división, y finalmente la suma.

---

## Ejemplos Completos

A continuación, se presentan algunos ejemplos completos que ilustran el uso de varias operaciones aritméticas juntas:

```
```python
# Suma, Resta, Multiplicación, y División
a = 10
b = 5
c = 2
resultado = (a + b - c) * c / b
print(resultado) # Resultado: 6.0
```

```
# Potencia y Módulo
a = 4
b = 2
c = 3
resultado = (a ** b) % c
print(resultado) # Resultado: 1
```

...

Estas operaciones son fundamentales para realizar cálculos y manipular datos numéricos en Python. Con una comprensión sólida de estos operadores, puedes desarrollar programas más complejos y eficientes.

Cadenas de Texto en Python

En Python, las cadenas de texto (strings) son secuencias de caracteres que se utilizan para representar texto. Las cadenas se delimitan utilizando comillas simples (') o dobles ("). A continuación, se explica cómo crear, manipular y concatenar cadenas de texto en Python, con ejemplos prácticos.

Creación de Cadenas de Texto

Para crear una cadena de texto en Python, se puede utilizar comillas simples o dobles. Ambas formas son equivalentes y permiten incluir caracteres especiales dentro de la cadena.

Ejemplos:

```
```python
cadena1 = 'Hola, mundo!'
cadena2 = "Python es increíble"
```
```

Si se necesita incluir comillas dentro de la cadena, se pueden usar comillas opuestas o el carácter de escape (`\`).

Ejemplos:

```
```python
cadena3 = "Esto es una 'cadena' con comillas simples dentro"
cadena4 = 'Esto es una "cadena" con comillas dobles dentro'
cadena5 = "Esto es una \"cadena\" con comillas dobles usando el carácter de escape"
```
```

Manipulación de Cadenas de Texto

Python ofrece varias formas de manipular cadenas de texto, como la indexación, el slicing y el uso de métodos de cadena.

Indexación

Cada carácter en una cadena tiene una posición específica (índice), que comienza desde 0 para el primer carácter y puede ser negativo para contar desde el final.

Ejemplos:

```
```python
cadena = "Python"
print(cadena[0]) # Resultado: 'P'
print(cadena[-1]) # Resultado: 'n'
```
```

Slicing

El slicing permite extraer una parte específica de una cadena utilizando la notación `[inicio:fin:paso]`.

Ejemplos:

```
```python
cadena = "Python"
print(cadena[0:2]) # Resultado: 'Py'
print(cadena[2:5]) # Resultado: 'tho'
print(cadena[:2]) # Resultado: 'Pto'
...

```

---

## Métodos de Cadenas

Python proporciona varios métodos integrados para trabajar con cadenas. Algunos de los métodos más comunes incluyen:

- `lower()`: Convierte todos los caracteres de la cadena a minúsculas.
- `upper()`: Convierte todos los caracteres de la cadena a mayúsculas.
- `replace(old, new)`: Reemplaza todas las apariciones de `old` con `new`.
- `split(delimiter)`: Divide la cadena en una lista de subcadenas, utilizando el delimitador especificado.
- `strip()`: Elimina los espacios en blanco al principio y al final de la cadena.

Ejemplos:

```
```python
cadena = " Python es Increíble "
print(cadena.lower()) # Resultado: ' python es increíble '
print(cadena.upper()) # Resultado: ' PYTHON ES INCREÍBLE '
print(cadena.replace("Increíble", "genial")) # Resultado: ' Python es genial '
print(cadena.split()) # Resultado: ['Python', 'es', 'Increíble']
print(cadena.strip()) # Resultado: 'Python es Increíble'
...

```

Concatenación de Cadenas

La concatenación de cadenas implica unir dos o más cadenas en una sola. Esto se puede hacer utilizando el operador `+` o el método `join()`.

Ejemplos:

```
```python
cadena1 = "Python"
cadena2 = "es genial"
resultado = cadena1 + " " + cadena2
print(resultado) # Resultado: 'Python es genial'

partes = ["Python", "es", "genial"]
resultado = " ".join(partes)
print(resultado) # Resultado: 'Python es genial'

```
```

Ejemplos prácticos

A continuación, se presentan algunos ejemplos prácticos que ilustran cómo trabajar con cadenas de texto en Python.

```
```python
Crear una cadena
saludo = "¡Hola, mundo!"

Manipulación de cadenas
print(saludo.lower()) # Resultado: '¡hola, mundo!'
print(saludo.upper()) # Resultado: '¡HOLA, MUNDO!'

Indexación y slicing
print(saludo[0]) # Resultado: '¡'
print(saludo[-1]) # Resultado: '!'
print(saludo[1:5]) # Resultado: 'Hola'

Concatenación de cadenas
nombre = "Python"
frase = nombre + " es genial"
print(frase) # Resultado: 'Python es genial'

Uso de métodos de cadena
frase_modificada = frase.replace("genial", "asombroso")
print(frase_modificada) # Resultado: 'Python es asombroso'

```
```

Estas técnicas y ejemplos proporcionan una base sólida para trabajar con cadenas de texto en Python, permitiendo realizar una amplia variedad de operaciones y manipulaciones.

Material complementario:

[Repositorio Python](#)

[Extreme Programming. Qué es y cómo usarlo](#) | Marvin López Mendoza

[Pipenv](#) | Ken Reitz

[Variables numéricas](#) | Nicolás Perez

[Operaciones](#) | Nicolás Perez

[Variables de Texto](#) | Nicolás Perez

Google Colab

[Tutorial Google Colab](#)

[Ejemplos de clase](#)

Te compartimos una [Guía de ejercicios complementarios](#). Ten presente que estas actividades no se entregan ni tienen corrección.